

1. Use el método de substitución para determinar la solución a la siguiente recurrencia:

$$T(n) = 4T\left(\frac{n}{2}\right) + n.$$

La solución de acuerdo con el *Master Method* es $\Theta(n^2)$, pero usar la hipótesis cn^2 falla. Realice el procedimiento bajo esa hipótesis para comprobar que falla y luego modifique la hipótesis para que funcione.

Solución:

1.1 Intento con hipótesis $T(n) \leq cn^2$

Asumimos que $T(k) \leq ck^2$ para todo $k < n$. Sustituyendo en la recurrencia:

$$T(n) = 4T(n/2) + n$$

$$T(n) \leq 4c(n/2)^2 + n$$

$$T(n) \leq 4c(n^2/4) + n$$

$$T(n) \leq cn^2 + n$$

Para que la inducción sea válida, se requiere que $cn^2 + n \leq cn^2$, lo cual implica $n \leq 0$. Como estamos analizando el crecimiento para $n \geq 1$, la hipótesis **falla**.

1.2 Hipótesis modificada: $T(n) \leq cn^2 - dn$

Para compensar el término lineal sobrante, asumimos $T(k) \leq ck^2 - dk$:

$$T(n) \leq 4(c(n/2)^2 - d(n/2)) + n$$

$$T(n) \leq 4(cn^2/4 - dn/2) + n$$

$$T(n) \leq cn^2 - 2dn + n$$

Queremos demostrar que $cn^2 - 2dn + n \leq cn^2 - dn$. Simplificando la desigualdad:

$$-2dn + n \leq -dn$$

$$n \leq dn$$

$$d \geq 1$$

La desigualdad se cumple para cualquier $d \geq 1$. Por lo tanto, se confirma que $T(n) = \Theta(n^2)$.

2. Resuelva la recurrencia

$$T(n) = 3T(\sqrt{n}) + \log_2 n.$$

Para hacerlo demuestre primero que se puede convertir en

$$S(m) = 3S\left(\frac{m}{2}\right) + m;$$

y luego resuelva esta recurrencia con el método de substitución. Con este resultado pruebe la respuesta para la recurrencia original.

Hint: note que, en $S(m)$, m parece ocupar el lugar que $\log_2 n$ tiene en $T(n)$.

Solución:

2.1 Cambio de variable

Sea $n = 2^m$, lo que implica que $m = \log_2 n$ y $\sqrt{n} = 2^{m/2}$. Sustituimos en la recurrencia original:

$$T(2^m) = 3T(2^{m/2}) + m$$

Definimos una nueva función $S(m) = T(2^m)$, obteniendo la forma:

$$S(m) = 3S(m/2) + m$$

2.2 Método de substitución para $S(m)$

Proponemos como hipótesis (dada la forma de la relación de recurrencia) $S(k) \leq ck^{\log_2 3} - dk$ para compensar el término lineal m :

$$S(m) \leq 3(c(m/2)^{\log_2 3} - d(m/2)) + m$$

$$S(m) = 3\left(c\frac{m^{\log_2 3}}{3} - \frac{dm}{2}\right) + m$$

$$S(m) = cm^{\log_2 3} - \frac{3}{2}dm + m$$

Buscamos que $cm^{\log_2 3} - \frac{3}{2}dm + m \leq cm^{\log_2 3} - dm$:

$$-\frac{3}{2}dm + m \leq -dm$$

$$m \leq \frac{1}{2}dm$$

$$d \geq 2$$

Esto demuestra que $S(m) = O(m^{\log_2 3})$.

2.3 Regreso a la variable original

Dado que $S(m) = T(n)$ y $m = \log_2 n$:

$$T(n) = \Theta((\log_2 n)^{\log_2 3}) \approx \Theta((\log_2 n)^{1.58})$$

3. **Use un árbol de recursión** para proveer una cota ajustada a la recurrencia

$$T(n) = T(n - a) + T(a) + cn,$$

donde $a \geq 1$, $c > 0$; ambas constantes. Puede suponer que n es múltiplo de a .

Solución:

Para analizar esta recurrencia mediante un árbol de recursión, observemos primero la estructura de las llamadas recursivas:

- En cada nivel, la recurrencia hace dos llamadas: $T(n - a)$ y $T(a)$
- El costo no recursivo en cada nivel es cn
- La llamada $T(a)$ es constante y llega a la base rápidamente
- La llamada $T(n - a)$ reduce el problema en a unidades

Construcción del árbol:

Nivel 0:	cn	Costo: cn
	$\swarrow \searrow$	
Nivel 1:	$c(n-a) \quad T(a)$	Costo: $c(n-a) + T(a)$
	$\swarrow \searrow$	
Nivel 2:	$c(n-2a) \quad T(a)$	Costo: $c(n-2a) + T(a)$
	$\swarrow \searrow$	
Nivel 3:	$c(n-3a) \quad T(a)$	Costo: $c(n-3a) + T(a)$
	$\dots$	
Nivel k:	$T(a) \quad (\text{varios } T(a))$	Costo base

Análisis por niveles:

- **Nivel 0:** costo = cn
- **Nivel 1:** costo = $c(n - a) + T(a)$
- **Nivel 2:** costo = $c(n - 2a) + T(a)$
- **Nivel i:** costo = $c(n - ia) + T(a)$

El árbol tiene profundidad $k = \frac{n}{a}$ (ya que en cada nivel reducimos a hasta llegar a la base).

Suma total de costos:

$$T(n) = \sum_{i=0}^{n/a-1} [c(n - ia) + T(a)]$$

Separando la suma:

$$T(n) = \sum_{i=0}^{n/a-1} c(n - ia) + \sum_{i=0}^{n/a-1} T(a)$$

Para la primera suma:

$$\sum_{i=0}^{n/a-1} c(n - ia) = c \sum_{i=0}^{n/a-1} (n - ia) = c \left[n \cdot \frac{n}{a} - a \sum_{i=0}^{n/a-1} i \right]$$

Usando $\sum_{i=0}^{k-1} i = \frac{k(k-1)}{2}$ con $k = \frac{n}{a}$:

$$= c \left[\frac{n^2}{a} - a \cdot \frac{\frac{n}{a}(\frac{n}{a} - 1)}{2} \right] = c \left[\frac{n^2}{a} - \frac{n(n-a)}{2a} \right]$$

$$= c \left[\frac{2n^2 - n(n-a)}{2a} \right] = c \left[\frac{2n^2 - n^2 + na}{2a} \right] = c \left[\frac{n^2 + na}{2a} \right] = \frac{cn^2}{2a} + \frac{cn}{2}$$

Para la segunda suma:

$$\sum_{i=0}^{n/a-1} T(a) = \frac{n}{a} \cdot T(a) = \Theta(n)$$

(ya que $T(a)$ es una constante)

Conclusión:

$$T(n) = \frac{cn^2}{2a} + \frac{cn}{2} + \Theta(n) = \Theta(n^2)$$

Por lo tanto, la cota ajustada para esta recurrencia es $T(n) = \Theta(n^2)$.

4. **Use el Master Method (si es posible)** para dar cotas ajustadas a las siguientes recurrencias:

a.

$$T(n) = 2T\left(\frac{n}{4}\right) + \sqrt{n}$$

Solución:

Para aplicar el Master Theorem, identificamos los parámetros de la recurrencia en la forma:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

En este caso:

- $a = 2$ (número de subproblemas)
- $b = 4$ (factor de reducción del tamaño)
- $f(n) = \sqrt{n} = n^{1/2}$ (costo de dividir/combinar)

Paso 1: Calculamos $n^{\log_b a}$:

$$n^{\log_b a} = n^{\log_4 2} = n^{1/2}$$

(porque $\log_4 2 = \frac{\log 2}{\log 4} = \frac{\log 2}{2 \log 2} = \frac{1}{2}$)

Paso 2: Comparamos $f(n)$ con $n^{\log_b a}$:

$$f(n) = n^{1/2} \quad \text{vs} \quad n^{\log_4 2} = n^{1/2}$$

Son iguales: $f(n) = \Theta(n^{\log_b a})$

Paso 3: Aplicamos el **Caso 2** del Master Theorem:

Cuando $f(n) = \Theta(n^{\log_b a})$, la solución es:

$$T(n) = \Theta(n^{\log_b a} \log n) = \Theta(n^{1/2} \log n) = \Theta(\sqrt{n} \log n)$$

Respuesta: $T(n) = \Theta(\sqrt{n} \log n)$

b.

$$T(n) = 4T\left(\frac{n}{2}\right) + n^2 \log_2 n$$

Solución:

Identificamos los parámetros:

- $a = 4$ (número de subproblemas)
- $b = 2$ (factor de reducción)
- $f(n) = n^2 \log_2 n$ (costo adicional)

Paso 1: Calculamos $n^{\log_b a}$:

$$n^{\log_b a} = n^{\log_2 4} = n^2$$

Paso 2: Comparamos $f(n)$ con $n^{\log_b a}$:

$$f(n) = n^2 \log_2 n \quad \text{vs} \quad n^{\log_2 4} = n^2$$

Aquí vemos que:

$$f(n) = n^2 \log_2 n = n^{\log_b a} \cdot \log n = \Theta(n^{\log_b a} \log n)$$

Esto no corresponde exactamente a ninguno de los tres casos básicos del Master Theorem en su forma estándar. Sin embargo, existe una **extensión del Caso 2** para cuando $f(n) = \Theta(n^{\log_b a} \log^k n)$ con $k \geq 0$.

Caso 2 Extendido: Si $f(n) = \Theta(n^{\log_b a} \log^k n)$ para algún $k \geq 0$, entonces:

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

En nuestro caso, $k = 1$, por lo tanto:

$$T(n) = \Theta(n^2 \log^{1+1} n) = \Theta(n^2 \log^2 n)$$

Respuesta: $T(n) = \Theta(n^2 \log^2 n)$

Nota: Si el Master Theorem básico no permite esta forma, podemos verificar usando el árbol de recursión o el método de sustitución para confirmar que efectivamente $T(n) = \Theta(n^2 \log^2 n)$.

5. **Dé una recurrencia** que cumpla con las condiciones del tercer caso del *Master Method* excepto la condición de regularidad.

Solución:

Se propone la recurrencia:

$$T(n) = T\left(\frac{n}{2}\right) + n(2 - \cos n)$$

con $a = 1$, $b = 2$, $f(n) = n(2 - \cos n)$.

Condición de dominancia (Caso 3): Como $2 - \cos n \geq 1$, se tiene $f(n) \geq n = \Omega(n^{\log_2 1+1})$. La condición se cumple con $\varepsilon = 1$.

Condición de regularidad: Se requiere $f(n/2) \leq c \cdot f(n)$ para algún $c < 1$. Evaluando en $n = 2k\pi$ con k impar:

- $\cos(2k\pi) = 1 \Rightarrow f(n) = n$
- $\cos(k\pi) = -1 \Rightarrow f(n/2) = \frac{n}{2}(3) = \frac{3n}{2}$

Esto da $f(n/2) = \frac{3n}{2} > f(n) = n$, por lo que ninguna constante $c < 1$ puede acotar la razón para todo n suficientemente grande. La condición de regularidad no se cumple.

6. Sea $G = (V, E)$ un grafo dirigido. Deseamos determinar si existe un camino que conecte a dos nodos $u, v \in V$; esto se conoce como el problema de conectividad-st o STCON. El algoritmo de Savitch, presentado a continuación, determina si existe un camino con tamaño máximo 2^i entre dos nodos u, v del grafo G :

Solución:

Sea $n = |V|$.

Divide: El camino de longitud $\leq 2^i$ se divide en dos mitades eligiendo un vértice intermedio w : hay camino de u a w de longitud $\leq 2^{i-1}$, y de w a v de longitud $\leq 2^{i-1}$.

Conquer: Se resuelven recursivamente $R(G, u, w, i-1)$ y $R(G, w, v, i-1)$ para cada $w \in V$.

Combine: Si ambas llamadas retornan $\top$ para algún w , se retorna $\top$; si ningún w funciona, se retorna $\perp$.

Recurrencia: En el peor caso se prueban los n vértices y se hacen $2n$ llamadas recursivas:

$$T(i) = 2n \cdot T(i-1) + \Theta(n), \quad T(0) = \Theta(1)$$

Expandiendo:

$$T(i) = (2n)^k T(i-k) + \Theta(n) \sum_{j=0}^{k-1} (2n)^j$$

Con $k = i$: $T(i) = O((2n)^i)$.

Para $i = \log_2 n$:

$$T(\log_2 n) = O((2n)^{\log_2 n}) = O(n \cdot n^{\log_2 n}) = O(n^{1+\log_2 n})$$

Como el exponente $1 + \log_2 n$ crece sin cota, este tiempo es superpolinomial: el algoritmo es ineficiente en tiempo. Su valor práctico es el uso de espacio $O(\log^2 n)$, que sí es eficiente.